LearnPath — Technical & Design Document

A Water-Cycle learning diagnostic powered by the Jev decision model. Grade 5 · teacher-facing · Next.js 16 · React 19 · Tailwind v4

> **One line:** A low score tells us a student is *struggling*. It doesn't tell us *what help they need*. LearnPath gathers targeted evidence, uses **Jev** (a System-1 decision model) to interpret it, and lets **explicit, testable code** choose the next step — so the same 40% can lead to different, inspectable decisions.

---

0. Table of contents

1. The core idea
2. System architecture (the "understanding layer")
3. Module-by-module walkthrough
4. The Jev judgments ("agents") in detail
5. The decision policy — Jev informs, code decides
6. The understanding profile (propensity vector) + cohort view
7. Accuracy: what we measure, and how to raise it
8. Why Jev over a general LLM (+ the four TypeSafe patterns we use)
9. Advantages & use cases
10. Limitations & guardrails
11. How to run / wire the key / demo script
12. Roadmap: from demo to edtech platform

---

1. The core idea

Two students can both score **40%** on the same water-cycle assessment for completely different reasons:

Student	Why they scored 40%	Right next step
Aarav	Doesn't grasp **evaporation** (a prerequisite)	Prerequisite revision
Meera	Knows the stages but can't **apply** them	Guided application
Rohan	Holds a **misconception** ("clouds fill like a sponge")	Alternative explanation
Sana	We only have a **score** — no evidence	Ask a diagnostic question
Dev	**Conflicting** evidence	Flag for teacher review

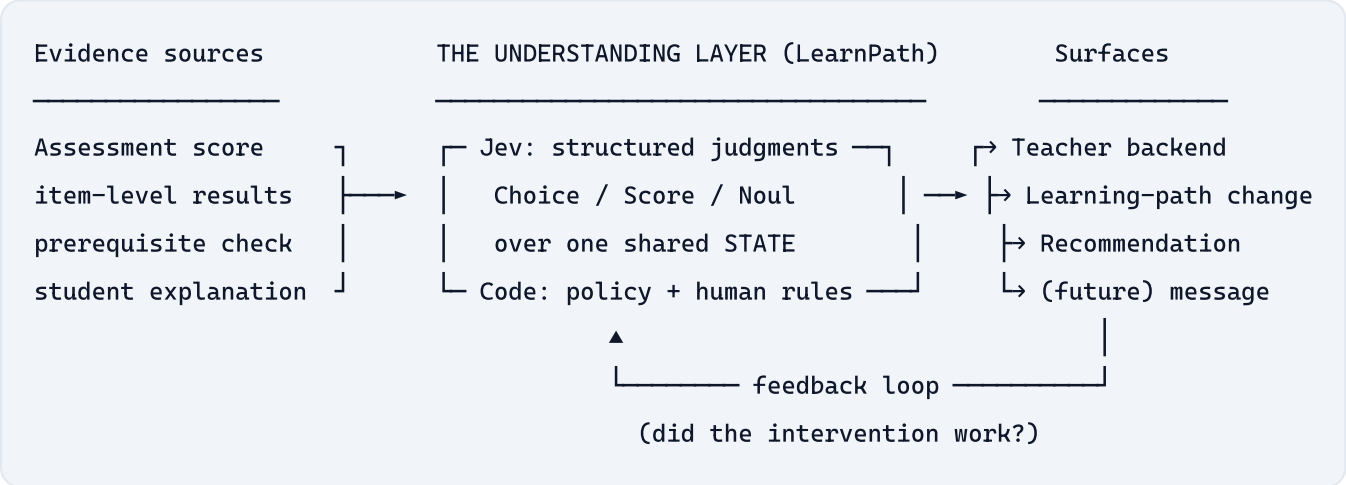
Giving all five the same remedial lesson wastes the opportunity. LearnPath's job is to decide **what to teach next**, from the evidence available **and the uncertainty in that evidence**.

The design discipline (straight from the Jev talk): **the LLM/decision model writes judgments; a "smart if-statement" — our own tested code — makes the final decision.** Jev interprets the evidence; LearnPath selects the intervention using explicit rules and never lets a model's guess silently override a directly-observed result.

---

2. System architecture — the "understanding layer"

LearnPath is not another LMS. It sits **between** the evidence sources and the actions taken on a student's behalf:



Two clean responsibilities:

- **Jev** answers *bounded* questions and returns **probabilities**, not prose, at low latency (~300 ms).
- **Our code** applies thresholds, prerequisite overrides, contradiction checks, and human-review rules to produce an **inspectable** decision with reason codes.

Request/response data flow (diagnostic path)

```
Browser (page.tsx)
  └─ POST /api/recommend { studentId }
       └─ lib/learnpath.ts: recommend(evidence)
            ├─ buildQuestions()              → 4 typed Jev questions
            ├─ lib/jev.ts: decide(state, qs)  → POST api.typesafe.ai/v1/systemone
            │        (key read server-side only)
            └─ decideRecommendation(evidence, jevAnswers)  → deterministic policy
       ←─ { recommendation: { intervention, reasons, needsTeacherReview, jev:{…} } }
```

---

3. Module-by-module walkthrough

3.1 `lib/jev.ts` — provider-agnostic Jev client (server-only)

The single place that talks to Jev. Responsibilities:

- **Provider abstraction.** Jev is reachable through several providers with slightly different endpoints. A `JEV_PROVIDER` env var selects one:

provider	endpoint	model	boolean type
`typesafe` *(our key)*	`https://api.typesafe.ai/v1/systemone`	`jev-latest`	`noul`
`vercel`	`https://ai-gateway.vercel.sh/v1/evaluate`	`typesafe-ai/jev`	`boolean`
`openrouter`	`https://openrouter.ai/api/alpha/decisions`	`typesafe/jev-1.13`	`noul`
`aimlapi`	`https://api.aimlapi.com/v1/decisions`	`typesafe/jev`	`noul`

- **Provider-neutral question type.** Callers write questions once (`kind: "boolean" | "choice" | "score"`); the client serializes them per provider (e.g. maps `boolean` → `noul` for TypeSafe).
- **Response normalization.** Every provider's answer is coerced into one uniform shape. Notably, TypeSafe returns a boolean as `{ "type": "noul", "noul": 0.79 }` — the normalizer reads `noul` , `probability` , `p` , or `yes` , whichever is present.
- **Security.** The API key is read from `JEV_API_KEY` / `AI_GATEWAY_API_KEY` **server-side only** and never shipped to the browser.

- **Typed errors.** `JevNotConfiguredError` (no key) and `JevRequestError` (non-200) let callers fall back to a clearly-labeled demo.

Request shape sent to Jev:

```
{
  "model": "jev-latest",
  "state": { /* the StudentEvidence object */ },
  "questions": {
    "likely_need":   { "type": "choice", "instructions": "…", "criteria": { … } },
    "enough_evidence": { "type": "noul", "instructions": "…" }
  }
}
```

Response (normalized):

```
{
  "likely_need":   { "kind": "choice",  "choice": "prerequisite_gap", "probabilities": { …
  "enough_evidence": { "kind": "boolean", "probability": 0.79 }
}
```

3.2 `lib/data.ts` — pure domain model & types (client + server safe)

No I/O, no secrets, so both the server and the client page import it.

- **Topic & skills.** `water_cycle`, with four skills ordered prerequisite→application: `evaporation_basics`, `condensation`, `precipitation`, `apply_real_world`.
- `StudentEvidence` — the input state: score, per-item results, a prerequisite check, the student's explanation, and an applied-task result.
- **Interventions** — four pre-written, teacher-controlled activities (`prerequisite_revision`, `alternative_explanation`, `guided_application`, `ask_diagnostic`), each with steps, a next activity, and a mastery check.
- **Seed students** — seven cases covering every scenario (5 strugglers + Priya "secure" + Kabir "progressing").
- **Profile helpers** — `avgMastery`, `weakestSkill`, `strongestSkill`, `isSecure`, `deriveProfileFlags`, `buildProfileSummary` (pure functions used by the cohort view).

3.3 `lib/learnpath.ts` — the brain (server-only)

- `buildQuestions()` — the 4 **diagnostic** Jev questions.

- `decideRecommendation(evidence, jev)` — the **deterministic policy** (Section 5).
- `recommend(evidence)` — full pipeline: build → call Jev → decide.
- `buildProfileQuestions()` — the 7 **understanding-profile** Jev questions.
- `computeProfile(evidence, jev)` / `profileFor(evidence)` — build the propensity vector.
- `demoJev` / `demoProfileJev` — clearly-labeled heuristic fallbacks, used only when no key is configured (or a live call fails). **Never presented as real Jev output.**

3.4 API routes

- `POST /api/recommend` — one student's diagnostic. Falls back to demo (labeled) if unconfigured or on error, surfacing the real error so the integration can be fixed.
- `GET /api/cohort` — every student's understanding profile, computed in parallel (`Promise.all`); per-student fallback to demo so one failure never blanks the dashboard.

3.5 UI

- `app/page.tsx` — two tabs: **Diagnostic** (single student, the "same 40%" hook) and **Teacher cohort**.
- `app/cohort-view.tsx` — the dashboard: a summary strip (at-risk / needs-evidence / ready counts), a roster of student cards (radar + mini-stats + flags), and a detail panel (large radar, mastery bars, propensity, raw Jev Choice).
- `components/charts.tsx` — dependency-free SVG `RadarChart`, `Bar`, `LabeledBar`, theme-aware via Tailwind fill/stroke utilities.

---

4. The Jev judgments ("agents") in detail

Each "agent" is a bounded Jev question. Several are answered **in parallel against one shared state** in a single request — this is Jev's core pattern and the reason latency stays low.

Diagnostic set (4 questions → `/api/recommend`)

id	type	what it asks	why Jev (not an LLM)
`likely_need`	**Choice**	prerequisite_gap / conceptual_misunderstanding / application_difficulty / insufficient_evidence	one label from a fixed set, with per-option probabilities — no hallucinated routing
`evidence_strength`	**Score**	how strongly the evidence supports that need (very_weak…very_strong)	an ordered rubric, returned as an interpolated score + per-rung probabilities
`enough_evidence`	**Noul**	enough to prescribe without asking more?	a calibrated yes-probability we can threshold on
`serious_prerequisite_gap`	**Noul**	must a prerequisite be fixed first?	a guard that feeds the "prerequisite-first" rule

Understanding-profile set (7 questions → `/api/cohort`)

id	type	what it asks
`likely_need`	Choice	same 4-way need
`mastery_evaporation_basics`	Score	mastery of evaporation (none…secure)
`mastery_condensation`	Score	mastery of condensation
`mastery_precipitation`	Score	mastery of precipitation
`mastery_apply_real_world`	Score	mastery of real-world application
`ready_to_advance`	Noul	ready for the next skill?
`at_risk`	Noul	at risk of falling behind without support?

The four mastery Scores become the **mastery radar**; the two Nouls become the **propensity meters**. Together they are the student's **propensity vector** — the "255 probabilities" idea from the talk, applied to learning.

4.3 Where Jev is used (the only two call sites)

Jev is called in **exactly two places** — everything else in the app is deterministic code. Each call is a **single batched request** (a fan-out):

Call site	Entry point	Jev questions	TypeSafe patterns used
Single-student diagnostic	`recommend()` → `POST /api/recommend`	4 (Choice + Score + 2 Noul)	Fan-Out · Intent Routing · Confidence-Gated Routing
Cohort understanding profile	`profileFor()` → `GET /api/cohort`	7 (Choice + 4 Score + 2 Noul)	Fan-Out · Composite Scoring · Confidence-Gated Routing

How we leverage it: Jev produces the **signals** (bounded labels, ordered scores, yes-probabilities); our code produces the **decision** (routing, thresholds, overrides, reason codes). We never ask Jev to "pick the intervention" in prose — it answers small, testable questions and `decideRecommendation()` composes them. (The named patterns are detailed in Section 8.)

---

5. The decision policy — *Jev informs, code decides*

`decideRecommendation()` turns Jev's probabilities into a defensible decision through **explicit, ordered rules**. This is the heart of the system and the reason a teacher can trust it.

1. **Thin evidence → never prescribe.** If we have ≤1 real evidence source beyond the score, *or* Jev's `enough_evidence` probability < **0.50**, force `insufficient_evidence` → *ask a diagnostic question*. (A single score is not a diagnosis.)
2. **Observed prerequisite failure overrides a model guess.** If the prerequisite check score < **40**, route to prerequisite revision even if Jev chose something else — a directly-observed fail beats an inference.
3. **Contradiction → collect more + flag.** If Jev says "prerequisite gap" but the prerequisite check is strong (≥ **70**), that's a contradiction → `insufficient_evidence` + teacher review.
4. **Conflicting signals → flag.** Strong prerequisite (≥70) but ≥2 downstream skills failed *and* no informative explanation → flag for review (don't prescribe blindly).
5. **Confidence gating.** If evidence strength < "moderate", or the top Choice probability < **0.60**, mark `needsTeacherReview`.

Every rule that fires appends a **reason code**, so the output is an audit trail, not a black box. Thresholds (`0.50` , `0.60` , `40` , `70` , strength `< 2`) are **tunable constants** — see Section 7.

---

6. The understanding profile & cohort view

For each student we compute:

- `mastery` **vector** (0–1 per skill) = Jev Score ÷ (levels−1).
- `readyToAdvance`, `atRisk` = Jev Noul probabilities.
- **Flags** (code-derived): *Ready to advance* (secure), *At risk*, *Needs evidence*, *Low mastery*, *On track*.
- `isSecure` heuristic — a student with high mastery + high readiness + low risk is "secure/ready," read from the mastery/propensity signals rather than the 4-way Choice (which has no "on-track" option and would otherwise mislabel a top student as "insufficient_evidence").
- **Plain-language summary** — templated from strongest/weakest skill, need, and risk.

The cohort dashboard then renders graphs + text together: a summary strip, radar-chart cards per student, and a detailed profile panel.

---

7. Accuracy — what we measure, and how to raise it

7.1 This is a *decision system*, not a classifier with ground truth

A student's "true" learning need is latent — there's no gold label to score against the way you would a spam classifier. So "accuracy" here is **a bundle of measures**, not a single number:

Measure	Definition	Why it matters
Scenario agreement	% of expert-labeled test cases where the decision matches the expected intervention	catches obvious regressions (we have 5–7 such cases)
Calibration	when Jev says $p = 0.8$, is it right ~80% of the time? (reliability curve / Brier score, ECE)	a probability is only useful if it's calibrated
Safe-abstention rate	% of genuinely ambiguous cases routed to *ask-diagnostic* / teacher review instead of a confident wrong answer	"knowing when not to prescribe" is a feature
Human-override rate	how often teachers change the recommendation	the real-world correctness signal
Downstream outcome	after the intervention, does the student pass a *new* transfer question?	the only measure that reflects learning, not just plausibility

> **Key point for judges:** success is **not** "Jev returns the same label every time." It's whether the app makes sensible distinctions, responds to evidence changes, and handles ambiguity responsibly. The follow-up transfer check is what actually proves the help landed.

7.2 How to increase the score (accuracy), in priority order

1. **Richer evidence, not a bigger prompt.** The biggest lever. Add item-level responses, short free-text explanations, and worked-example results. Jev reasons far better over a few hand-crafted signals than over one raw score. (From the talk: "create handcrafted features that make sense," don't dump raw data.)
2. **Sharper question criteria.** Spell out what each Score rung and each Choice option means. Ambiguous instructions → fuzzy probabilities. Define levels (none/emerging/developing/secure), don't ask for an unexplained number.
3. **Calibrate thresholds against labeled outcomes.** Treat `0.50`, `0.60`, `40`, `70` as hyper-parameters. Collect ~50–100 teacher-labeled cases, then tune thresholds to maximize agreement while keeping the abstention/override rates sane. **Jev's confidence ≠ probability the diagnosis is correct** — calibrate against your own data before trusting a cutoff.
4. **Decompose decisions into more, smaller judgments.** One hallucinated bit can't topple the whole decision when it's one of several bounded questions feeding deterministic code (vs. a single chain-of-thought answer that fails as a unit).
5. **Add a fallback stage.** On low-confidence Choice/Score, re-ask with a second model (decision fallback) or escalate to a human. We already support this pattern via teacher review.
6. **Close the feedback loop.** Record whether the follow-up transfer question is answered correctly, and feed the outcome back as new evidence — turning a one-shot classifier into an adaptive loop.
7. **Fine-tune your own decision model (CLM).** TypeSafe's open **CLM** (a frozen Qwen-3-8B backbone with trainable *state* and *action* heads) can be fine-tuned on your labeled cases and self-hosted — higher accuracy on your specific rubric, data stays internal, low marginal cost on existing GPU infra.

7.3 Pattern-aligned upgrades (the highest-leverage, docs-backed next steps)

Each of these maps directly onto a documented TypeSafe pattern (Section 8.1):

- **Per-action confidence gates** *(Confidence-Gated Routing)*. Today `needsTeacherReview` uses a single 0.60 floor. Upgrade to **stakes-based thresholds**: a low-stakes action (show a hint) can auto-act at ≥0.60; a high-stakes action (mark a skill *mastered* and skip ahead) should require ≥0.85; the **middle band asks the teacher to confirm**. This mirrors TypeSafe's banking example (balance vs. transfer) and is the single biggest reliability win.
- **Weighted composite mastery** *(Composite Scoring)*. `avgMastery` is equal-weighted. Switch to **decision-specific weights** — e.g. an "application-readiness" score weights `apply_real_world` highest: `readiness = Σ wᵢ·masteryᵢ`. Same Jev scores, different rankings per decision (exactly TypeSafe's resume IC-vs-EM example).
- **Speculative fan-out judgments** *(Speculative Fan-Out)*. Add misconception-specific Nouls to the single request — e.g. "*holds the 'clouds fill like a sponge' belief?*", "*thinks evaporated water is*

simply '*gone*'*?*". Code reads only the branch the Choice selects; the extra questions add diagnostic power at ~no latency cost.

---

8. Why Jev over a general LLM — and the four patterns we use

8.0 The one solid reason

A routing / triage decision needs two things from **one fast, cheap call**: a **bounded answer** (one of a fixed set, or a number on a fixed scale) and a **calibrated confidence** in that answer.

A general LLM is a *next-token predictor*. To get a decision out of it you must force it onto a label, parse free text, and trust a "confidence" it was **never trained to calibrate** — and you pay System-2 latency and cost for every judgment. **Jev is a System-1 decision model: the output *is* the decision, with per-option probabilities you can threshold.**

That single property — **bounded output + calibrated confidence, per call** — is what makes confidence-gated routing, fan-out, and composite scoring *possible at all*. You cannot safely build those on raw LLM prose. That is the reason to choose Jev here, not a general LLM.

8.1 LearnPath implements all four documented TypeSafe patterns

From TypeSafe's own pattern catalog — LearnPath isn't using Jev ad-hoc; it composes four named patterns:

TypeSafe pattern	What it is	Where LearnPath uses it
Speculative Fan-Out	many typed questions over one shared `state` in a single call; code picks the relevant answers	both call sites send every question at once over the student `state`; `serious_prerequisite_gap` is a *speculative* question used only on some branches — "everything for the decision tree from one call"
Intent Routing	a Choice classifies intent → route to a handler (code / specialist / human)	`likely_need` Choice routes to one of four intervention handlers, or to *ask-diagnostic* / teacher (human escalation)
Confidence-Gated Routing	confidence is a *second axis*; a global floor + per-action thresholds decide auto-act vs. confirm vs. escalate	`enough_evidence` < 0.50 → ask for more; top Choice prob < 0.60 or weak evidence strength → teacher review; contradiction → escalate. Confidence decides *whether to act*, not just *what*
Composite Scoring	split a judgment into dimensions, atomic Scores, normalize ÷(levels−1), combine with code-controlled weights	the understanding profile: 4 per-skill mastery Scores → normalized → combined (`avgMastery` , weightable) into the propensity vector + flags

8.2 Why Jev, not a general LLM — the mechanics

Need	General LLM	**Jev**
One of a fixed set of answers	prompt-coax + parse; can drift off-list	**native Choice** over fixed criteria
A number you can threshold	asks for a number it may hallucinate	**Score** = interpolated position on a defined rubric
A calibrated yes/no probability	"confidence" is uncalibrated prose	**Noul** = a probability meant to be thresholded
Several judgments at once	N sequential calls (latency ×N)	**one fan-out call**, parallel, ~flat latency
Cheap at cohort scale	per-token, System-2 cost	input-token priced, ~300 ms
Keep workflow out of the model	model must "know" the branching	branching lives in code; the model answers atomically

8.3 Decision model vs. LLM vs. classical ML

	Jev (decision model)	**LLM (e.g. GPT/Claude)**	**Classical ML**
Output	probabilities over fixed options	free-form tokens	probabilities (needs training data)
System	**System 1** — fast reflex	System 2 — slow reasoning	fast once trained
Latency	~300 ms	seconds	ms
Hallucination risk	low (bounded output)	higher on routing/numbers	none, but brittle
Setup cost	write typed questions	write prompts	collect labels, train, serve
Explainability	per-option probabilities	reasoning text (can be post-hoc)	feature weights
Best at	**bounded, repeated decisions**	generation, empathy, language	large labeled tabular data

They aren't apples-to-apples. **LLM writes; Jev is a smart if-statement.** In LearnPath we use **Jev for the judgments** and reserve an **LLM for generation** (future: the student-facing message in the right tone).

8.4 "Naive Jev output" vs. LearnPath's layered decision

The critical comparison for this project: what do you get if you just trust Jev's raw output, versus running it through our policy layer?

	Raw Jev output alone	**LearnPath (Jev + deterministic policy)**
Ambiguous student (only a score)	returns *some* label with a probability	**abstains** → asks a diagnostic question
Contradiction (model vs. observed check)	may confidently pick the wrong need	**overrides** with the observed result + flags review
Auditability	a label + probabilities	label + probabilities **+ reason codes + override trail**
Teacher trust	"why did it say that?"	every decision is inspectable and reversible
Correctness over time	static	**improves** via the follow-up feedback loop

The layer is the product. Jev makes the layer *possible* (fast, bounded, calibrated signals); the layer makes Jev *safe to act on*.

9. Advantages & use cases

Advantages

- **Fast & cheap** per decision (~300 ms, input-token-priced) — viable at classroom/cohort scale.
- **Low hallucination** on the routing decision because outputs are bounded.
- **Inspectable** — reason codes + probabilities a teacher can audit.
- **Responsible by design** — abstains under uncertainty; keeps a human in the loop.
- **Portable** — provider-agnostic client; swap TypeSafe ↔ Vercel ↔ OpenRouter with one env var.

Use cases (edtech)

- Teacher cohort triage: who's at risk, who's ready, who needs more evidence — at a glance.
- Adaptive sequencing: bend the learning path per student instead of one remedial track.
- Early-warning flags for intervention before a student falls behind.
- Personalized, teacher-approved nudges (future, LLM-generated, Jev-gated).
- Plugs into existing LMS via standards (xAPI / Caliper events, LTI) — augment, don't replace.

---

10. Limitations & guardrails

- **Not a learning-assessment engine.** Jev can't establish true understanding from a percentage — it needs evidence (items, prerequisite checks, explanations).
- **Confidence ≠ correctness.** Calibrate thresholds against real outcomes; never auto-cast a non-zero probability to a verdict.
- **Minors' data.** Human-in-the-loop on anything student-facing; privacy/consent by design.
- **Small context.** Decision models have limited context — feed hand-crafted features, not raw dumps.
- **Demo mode is labeled.** When no key is set (or a live call fails), synthetic judgments are clearly marked "Demo (not real Jev)."

---

11. How to run / wire the key / demo script

Run (from `learnpath/`):

```
npm run dev
```

Open http://localhost:3000.

Wire the Jev key — put it in `learnpath/.env.local` (gitignored, server-side only):

```
JEV_API_KEY=apikey_…
JEV_PROVIDER=typesafe
```

Confirm a real call:

```
npm run verify:jev
```

(PowerShell note: use `;` or separate lines, not `&&`.)

90-second demo script

1. Student rail — point out the **same ~40%** across students.
2. **Aarav** → Recommend → prerequisite revision (failed evaporation check).
3. **Meera** (also 40%, strong prerequisite) → guided application. *Same score, different decision.*
4. **Sana** (only a score) → LearnPath **refuses to guess** and asks for more evidence.
5. Switch to **Teacher cohort** → the class at a glance: Priya *ready to advance*, the rest *at risk*, with mastery radars.

---

12. Roadmap — from demo to edtech platform

- **Ingest** real learning events via xAPI / Caliper; plug into an LMS via LTI.
- **Student-facing messages** — LLM-generated, tone-adapted, Jev-gated, teacher-approved.
- **Adaptive path engine** — a skill graph (DAG) whose transitions Jev + code re-route per student.
- **Outcome logging & calibration** — close the loop; tune thresholds on real data.
- **Self-hosted CLM** — fine-tune the decision model on your rubric for higher accuracy and data residency.

Jev returns structured probabilities (System 1). LearnPath's own code applies thresholds and human-review rules to make the final decision.